

AUTOMATIC GUN DETECTION AND RECOGNITION USING DEEP LEARNING

A Mini Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

By

ADDAGULLA ISHIKA	2456-22-733-002
CHOLLETI SRI VARSHINI	2456-22-733-303
GOPU SREEJA	2456-22-733-304

Under the guidance of

Dr. V HARIKA

Asst Professor(CSE)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Gokaraju Lailavathi Womens Engineering College
Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090

(2024-2025)

Department of Computer Science and Engineering
Gokaraju Lailavathi Womens Engineering College

Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090

(2024-2025)



CERTIFICATE

This is to Certify that A Mini Project report entitled “**Automatic Gun Detection And Recognition Using Deep Learning**” is being submitted by Addagulla Ishika(2456-22-733-002), Cholleti Sri Varshini (2456-22-733-303),Gopu Sreeja(2456-22-733-304) in partial fulfillment of the requirement of the award for the degree of Bachelor of Engineering in “Computer Science and Engineering” O.U., Hyderabad during the year 2023-2024 is a record of bonafide work carried out by them under my guidance. The results presented in this project have been verified and are found to be satisfactory.

Project Guide

Dr. V Harika

Assistant Professor,
Dept of CSE

H.O.D

Dr. Padmalaya Nayak

Professor & Head,
Dept. of CSE

External Examiner(s)



Department of Computer Science and Engineering
Gokaraju Lailavathi Womens Engineering College

Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090

(2024-2025)

DECLARATION

We, Addagulla Ishika bearing Ht.No. 2456-22-733-002, Cholleti Sri Varshini bearing Ht.No. 2456-22-733-303 and Gopu Sreeja bearing Ht.No. 2456-22-733-303 hereby certify that the minor project entitled “**AUTOMATIC GUN DETECTION AND RECOGNITION USING DEEP LEARNING**” is submitted in the partial fulfilment of the required for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

This is a record work carried out by us under the guidance of Dr.V HARIKA, Assistant Professor, CSE, Gokaraju Lailavathi Womens Engineering College, Nizampet. The results embodied in this report have not been reproduced/copied from any source. The results embodied in this report have not been submitted to any other university or institute for the award of any other degree or diploma.

Addagulla Ishika	2456-22-733-002
Cholleti Sri Varshini	2456-22-733-303
Gopu Sreeja	2456-22-733-304

ACKNOWLEDGEMENT

It is our privilege and pleasure to express my profound sense of respect, gratitude and indebtedness to our guide Dr. V Harika, Assistant Professor, Department of Computer Science and Engineering, Gokaraju Lailavathi Womens Engineering College, for her inspiration, guidance, cogent discussion, constructive criticisms and encouragement throughout this dissertation work.

We express our sincere thanks to Project Coordinator Mr Raj kumar, Assistant Professor, Department of Computer Science and Engineering, Gokaraju Lailavathi Womens Engineering College, for his valuable suggestions and constant helping in completing the work.

We express our sincere thanks to Dr. A. Sai Hanuman, Principal, Gokaraju Lailavathi Womens Engineering College, Nizampet, Hyderabad, for his encouragement and constant help.

We extend our sincere thanks to all the teaching and non-teaching staff of Computer Science and Engineering Department for their support and encouragement.

Last but not least, we wish to acknowledge my friends and family members for giving moral strength moral strength and helping us to complete this dissertation.

ABSTRACT

Real-time object detection to improve surveillance methods is one of the sought-after applications of Convolutional Neural Networks (CNNs). This research work has approached the detection of handguns in areas monitored by cameras . Gun violence and mass shootings are also on the rise in certain parts of the world. Such incidents are time sensitive and can cause a huge loss to life and property. Hence, the proposed work has built a deep learning model based on the YOLOv3 algorithm with the accuracy 98.98% that processes a video frame-by-frame to detect such anomalies in real-time and generate an alert for the concerned authorities.

.

CONTENTS

Chapter-1 INTRODUCTION.....	1
1.1 MOTIVATION.....	1
1.2 PROBLEM STATEMENT.....	1
1.3 PROJECT OBJECTIVE.....	1
Chapter-2 LITERATURE SURVEY	2
2.1 GUN DETECTION AND RECOGNITION CHALLENGES.....	2
2.2 YOLOV3(YOU LOOK ONLY ONCE VESRION3)	2
2.3 NON-MAXIMUM SUPPRESSION(NMS).....	3
2.4 OPTIMIZATION OF DISTANCE IN GUN RECOGNITION	3
2.5 COMPARISION OF YOLOV3 AND NON-MAXIMUM SUPPRESSION.....	3
2.6 CURRENT RESEARCH GAPS	3
2.7 CONCLUSION.....	4
Chapter-3 SOFTWARE REQUIREMENT SPECIFICATION.....	5
3.1 SOFTWARE REQUIREMENTS.....	5
3.2 HARDWARE REQUIREMENTS	5
3.3 FUNCTIONAL REQUIREMENTS.....	5

3.4 NON-FUNCTIONAL REQUIREMENTS	6
---------------------------------------	---

Chapter-4 SYSTEM DESIGN	7
--------------------------------------	----------

4.1 SYSTEM DESIGN.....	7
------------------------	---

4.2 PROPOSED METHODOLOGY	8
--------------------------------	---

4.3 UML DESIGN	9
----------------------	---

4.3.1 CLASS DIAGRAM.....	11
--------------------------	----

4.3.2 USE CASE DIAGRAM	12
------------------------------	----

4.3.3 COMPONENT DIAGRAM	13
-------------------------------	----

4.3.4 SEQUENCE DIAGRAM	14
------------------------------	----

4.3.5 ACTIVITY DIAGRAM.....	15
-----------------------------	----

4.4 TECHNOLOGY DESCRIPTION.....	16
---------------------------------	----

Chapter-5 IMPLEMENTATION	17
---------------------------------------	-----------

5.1 IMPLEMENTATION	17
--------------------------	----

5.2 MODULES	19
-------------------	----

5.3 EXECUTABLE CODE	22
---------------------------	----

Chapter-6 TESTING	25
--------------------------------	-----------

6.1 TESTING DEFINITION	25
------------------------------	----

6.2 UNIT TESTING.....	25
6.3 PERFORMANCE METRICS AND TEST RESULTS.....	25
Chapter-7 RESULTS	28
Chapter-8 CONCLUSION	30
8.1 CONCLUSION.....	30
8.2 FUTURE SCOPE	30
REFERENCES	31
PLAGIARISM TEST	32

SNO.	LIST OF FIGURES	PAGE NO.
4.2.1	Proposed Methodology	8
4.3.1	UML Hierarchy Diagrams	10
4.3.1.1	Class Diagram	11
4.3.2.1	Use Case Diagram	12
4.3.3.1	Component Diagram	13
4.3.4.1	Sequence Diagram	14
4.3.5.1	Activity Diagram	15
7.1	Gun detected	28
7.2	Gun detected	28
7.3	Gun detected	29
7.4	Gun detected	29

LIST OF TABLES

Table. No	Table Name	Page. No
6.3.1	Performance Metrics for YOLOv3 on Custom Pistol Dataset	27
6.3.2	Performance at Varying Distances	27

CHAPTER-1

INTRODUCTION

1.1 MOTIVATION

The rising incidents of gun violence and mass shootings across the globe highlight a critical need for faster and more reliable surveillance systems. Traditional methods often fail to respond in real-time, leading to delayed actions and increased casualties. This project aims to harness the power of deep learning and real-time object detection through the YOLOv3 model to identify handguns in surveillance footage instantly. By automating threat detection, we aim to minimize response time, enhance public safety, and support law enforcement with timely alerts, ultimately contributing to a safer environment.

1.2 PROBLEM STATEMENT

In today's world, the increasing number of gun-related crimes and mass shootings poses a serious threat to public safety. Traditional surveillance systems lack the capability to detect weapons in real-time, resulting in delayed responses and potential loss of life and property. There is a pressing need for an intelligent system that can automatically monitor live video feeds and instantly identify firearms, enabling quicker alerts and response actions. This project addresses the problem by developing a real-time handgun detection system using the YOLOv3 deep learning model, which can efficiently analyze video frames, detect handguns with high accuracy (98.98%), and notify concerned authorities to prevent or minimize the impact of violent incidents.

1.3 PROJECT OBJECTIVE

The objective of this project is to design and implement a real-time handgun detection system using the YOLOv3 algorithm, aimed at enhancing the effectiveness of surveillance methods. By leveraging Convolutional Neural Networks (CNNs), the system processes video feeds frame-by-frame to accurately detect the presence of handguns with an impressive accuracy of 98.98%. Upon detecting a Gun, the system instantly triggers an alert to notify the concerned authorities, enabling timely intervention and potentially preventing violent incidents. This automated approach reduces the need for constant human monitoring and serves as a valuable tool in strengthening public safety in areas such as schools, malls, transportation hubs, and other vulnerable locations.

CHAPTER 2

LITERATURE SURVEY

Several studies have explored object detection for surveillance using deep learning techniques. Early models like R-CNN and Faster R-CNN offered high accuracy but lacked real-time performance. YOLO (You Only Look Once) models addressed this issue by providing fast and accurate detection in a single pass. Among these, YOLOv3 has gained attention for its lightweight architecture and high precision in real-time applications. Previous research has shown the potential of such models in detecting weapons in images and videos. Building on this, our project uses YOLOv3 to detect handguns in live surveillance footage, aiming to improve response time and public safety.

2.1 GUN DETECTION AND RECOGNITION CHALLENGES

There are many difficulties in detecting guns using cameras and deep learning. Sometimes, guns are only **partly visible** because they are hidden under clothes or bags. In security footage, guns can also appear **very small**, making them hard to spot. Objects like **phones or tools** may look like guns, causing **wrong detections**. **Low light**, shadows, or fast movements can make the video blurry, which affects the model's performance. Also, detecting guns quickly in real-time needs **powerful systems**. Another problem is the lack of **real videos with guns** for training the model. Just detecting a gun is not always enough — we also need to know if someone is showing **suspicious behavior**, which is even more difficult.

2.2 YOLOV3(YOU LOOK ONLY ONCE VERSION 3)

YOLOv3 (You Only Look Once version 3) is a real-time object detection algorithm based on Convolutional Neural Networks (CNNs). It processes the entire image at once and predicts multiple bounding boxes along with class probabilities. YOLOv3 uses Darknet-53 as its backbone architecture, which consists of 53 convolutional layers and residual connections, making it both deep and efficient. The model performs detection at three different scales, which helps in identifying both small and large objects. Due to its speed, accuracy, and multi-scale detection, YOLOv3 is widely used in applications like gun detection in surveillance systems.

2.3 NON-MAXIMUM SUPPRESSION(NMS)

Non-Maximum Suppression (NMS) is a post-processing technique used in object detection to remove **duplicate or overlapping bounding boxes** for the same object. It selects the box with the **highest confidence score** and suppresses the others that have a high overlap with it. This helps in improving the **accuracy and clarity** of the final detection output, ensuring that each object is detected **only once**.

2.4 OPTIMIZATION OF DISTANCE IN GUN RECOGNITION

In gun recognition, optimizing the detection accuracy across different distances from the camera is an important challenge. Guns may appear very small in distant frames and larger when closer, which can affect the model's performance. To address this, the system uses multi-scale detection, as in YOLOv3, where objects are detected at three different scales. This helps in recognizing guns whether they are near or far from the camera. Additionally, image resizing, anchor box tuning, and confidence threshold adjustments are done to improve detection accuracy across varied distances.

2.5 COMPARISON OF YOLOV3 AND NON-MAXIMUM SUPPRESSION

Several studies have compared the performance of the Haar Cascade and LBP classifiers. The Haar Cascade classifier generally outperforms the LBP classifier in terms of accuracy and robustness, particularly in real-time applications. However, the LBP classifier is often favored for its computational efficiency and ability to handle different lighting conditions and facial expressions.

2.6 CURRENT RESEARCH GAPS

Although deep learning models like YOLOv3 have shown high accuracy in gun detection, several research gaps still exist. One major issue is the lack of real-world surveillance datasets containing diverse gun appearances, poses, and environments. Most existing datasets are staged or synthetic, which limits the model's performance in real scenarios. Additionally, detecting partially visible or concealed weapons remains a challenge, especially in crowded or low-light conditions. Another gap lies in combining object detection with behavior analysis, such as identifying suspicious movements or intent to use a weapon. Real-time detection with limited hardware (e.g., in CCTV systems) and ensuring low false positives are also ongoing concerns. Addressing these gaps is essential for developing more robust and practical gun detection systems.

2.7 CONCLUSION

The proposed system uses the **YOLOv3 deep learning algorithm** for real-time gun detection in surveillance videos. It can accurately identify handguns from video frames with an accuracy of around **98.98%**, which is higher than earlier models like traditional CNNs and YOLOv2. This shows that the system is both **fast and reliable** in detecting guns in real-world situations. It can be easily added to **CCTV cameras or smart surveillance systems**, helping improve public safety and enabling **early warning of threats**.

CHAPTER-3

SOFTWARE REQUIREMENT SPECIFICATION

This chapter gives an overview of the software and hardware components required for our project.

3.1 SOFTWARE REQUIREMENTS

Operating System : Windows 11

Coding Language : Python 3.12.2

Framework : Darknet-53

3.2 HARDWARE REQUIREMENTS

System : intel i3 or above

Storage : Sufficient storage

3.3 FUNCTIONAL REQUIREMENTS

These are the requirements that the end user specifically demands as essential capabilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the implementation. They are defined based on the inputs to the system, the operation performed, and the output expected. These are features that the user can see and interact with in the final system, unlike non-functional requirements which are more about the quality attributes.

- **Real-Time Weapon Detection (Pistol)**

The system shall detect pistols in real-time using the YOLOv3 deep learning algorithm.

- **Frame-by-Frame Video Processing**

The system shall process video input (live or file) frame-by-frame to analyze for weapon presence.

- **Visual Feedback of Detection**

Bounding boxes and confidence scores shall be overlaid on detected weapons and displayed in a video window.

- **Keyboard-Controlled Exit**

The user shall be able to stop the detection process by pressing a key (e.g., q).

- **Pretrained Model Integration**

The system shall load custom YOLOv3 configuration and weight files for inference.

- **Basic User Interface**

The system shall use OpenCV to display detections in a user-friendly window during runtime.

3.4 NON-FUNCTIONAL REQUIREMENTS

These are the quality constraints that the system must satisfy to ensure usability, reliability, and performance. Unlike functional requirements, these define how the system should behave rather than what it should do. They ensure that the delivered system meets user expectations beyond basic functionality.

- **Portability**

The system shall run on Windows 7 or later and require minimal hardware (Intel i3 processor, 8GB RAM).

- **Security**

While not currently implemented, the system design considers future enhancements for secure access control and configuration protection.

- **Maintainability**

The codebase shall be modular, allowing easy updates to the model, thresholds, and video input configuration.

- **Reliability**

The system shall handle common runtime issues such as missing video files or end-of-stream without crashing.

- **Scalability**

The architecture shall allow future upgrades to support multiple weapon classes and integration with alert or logging systems.

- **Performance**

The system shall process frames with minimal delay, achieving near real-time performance on CPU-based systems.

- **Flexibility**

System parameters (e.g., confidence threshold, model files, input resolution) shall be configurable for different deployment scenarios.

CHAPTER-4

4.1 SYSTEM DESIGN

Introduction

The system design phase involves translating the system requirements into a blueprint for constructing the software. It includes architectural design, data flow design, component design, and interface design. This chapter describes the structural components and interaction of modules for the weapon detection system using deep learning.

System Architecture

The architecture of the system is modular, allowing flexibility and scalability. The system accepts video input, performs real-time weapon detection using a YOLOv3 model, and displays the detection results to the user.

Main Components:

Input Module (Video/Camera)

Preprocessing Module

Detection Engine (YOLOv3)

Output Module (Display)

4.2 PROPOSED METHODOLOGY:

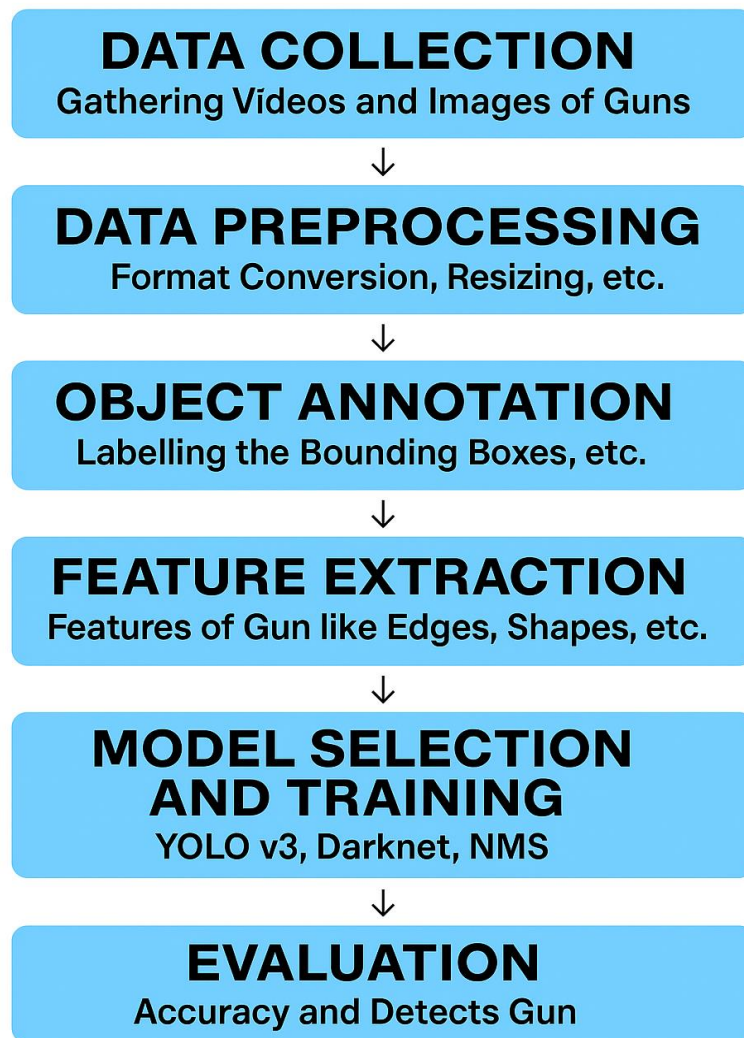


Figure-4.2.1 Proposed Methodology

4.3 UML Design:

Unified Modeling Language (UML) is a general purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed.

UML is not a programming language; it is rather a visual language. We use UML diagrams to portray the behavior and structure of a system, UML helps software engineers, businessmen and system architects with modeling, design and analysis. The Object Management Group (OMG) adopted Unified Modeling Language as a standard in 1997. It's been managed by OMG ever since. International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

Do we really need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non programmer's essential requirements, functionalities and processes of the system.
- UML is linked with object oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Types of UML Diagrams:

❑ Structural Diagrams:

Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.

❑ Behavior Diagrams:

Capture dynamic aspects or behavior of the system. Behavior diagrams include: Use Case Diagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

The image below shows the hierarchy of diagrams according to UML

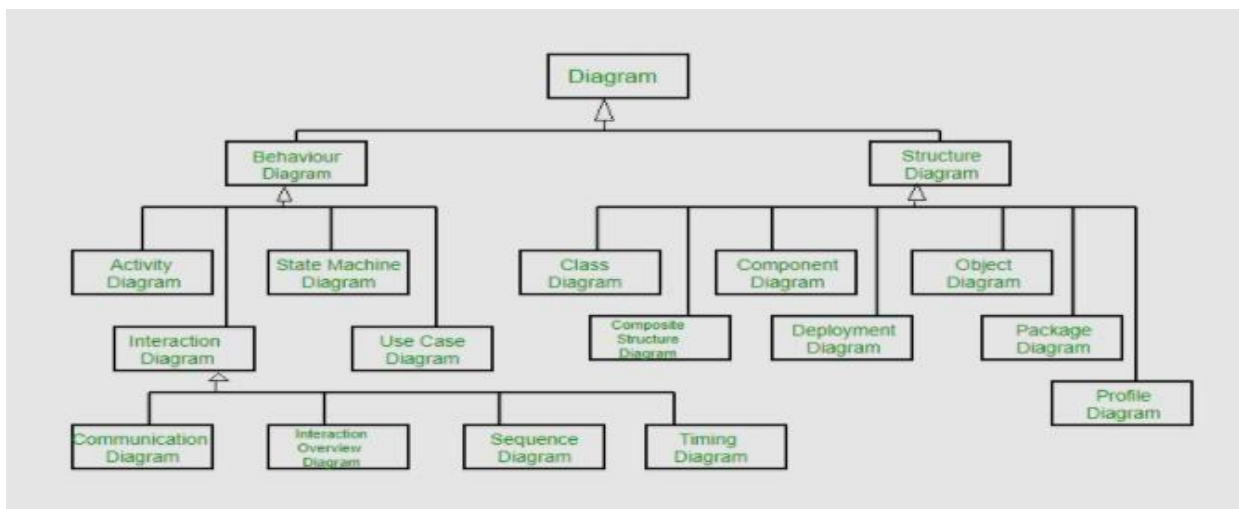


Figure-4.3.1 UML Hierarchy diagrams

4.3.1 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

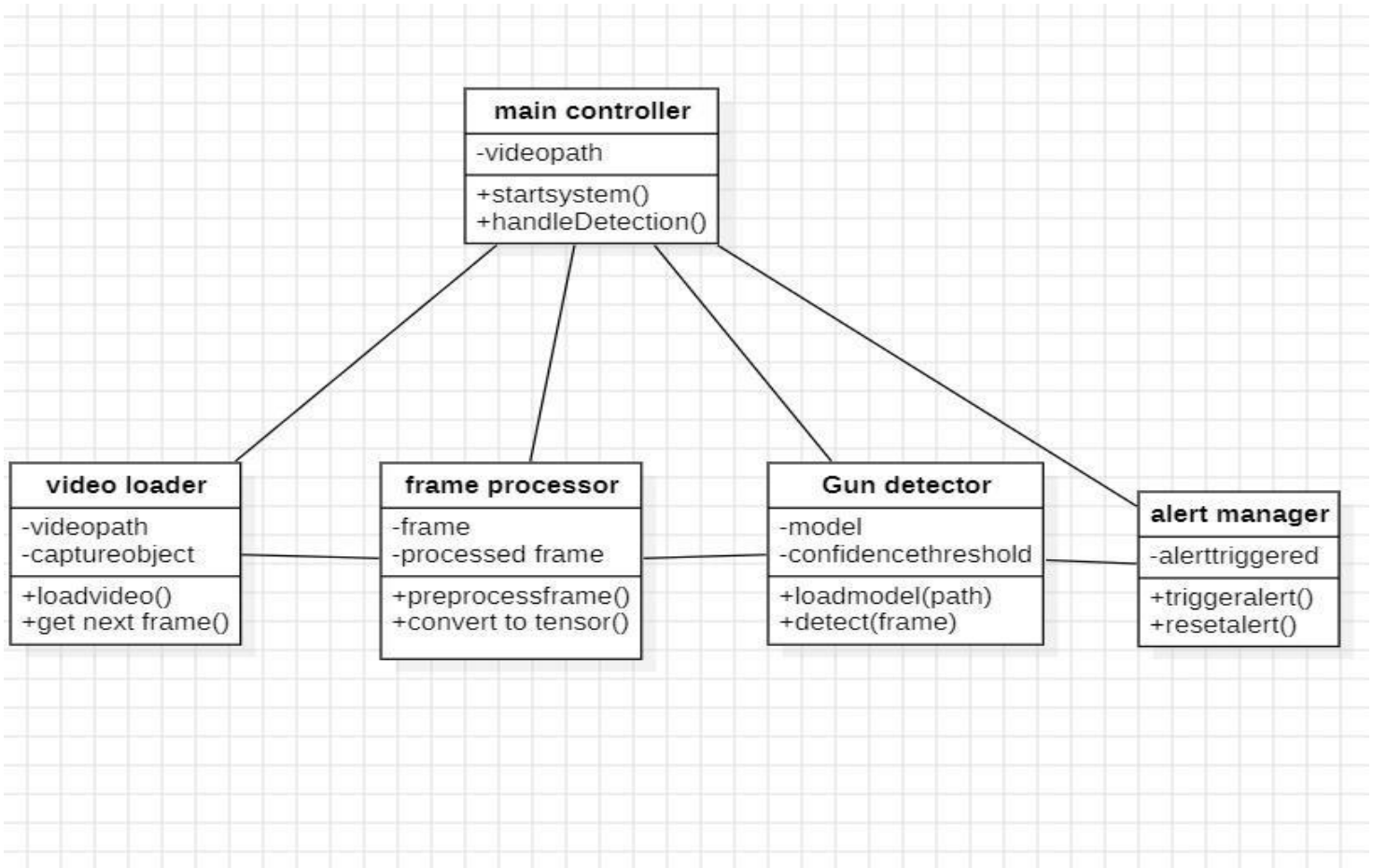


Figure-4.3.1.1 Class Diagram

4.3.2 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

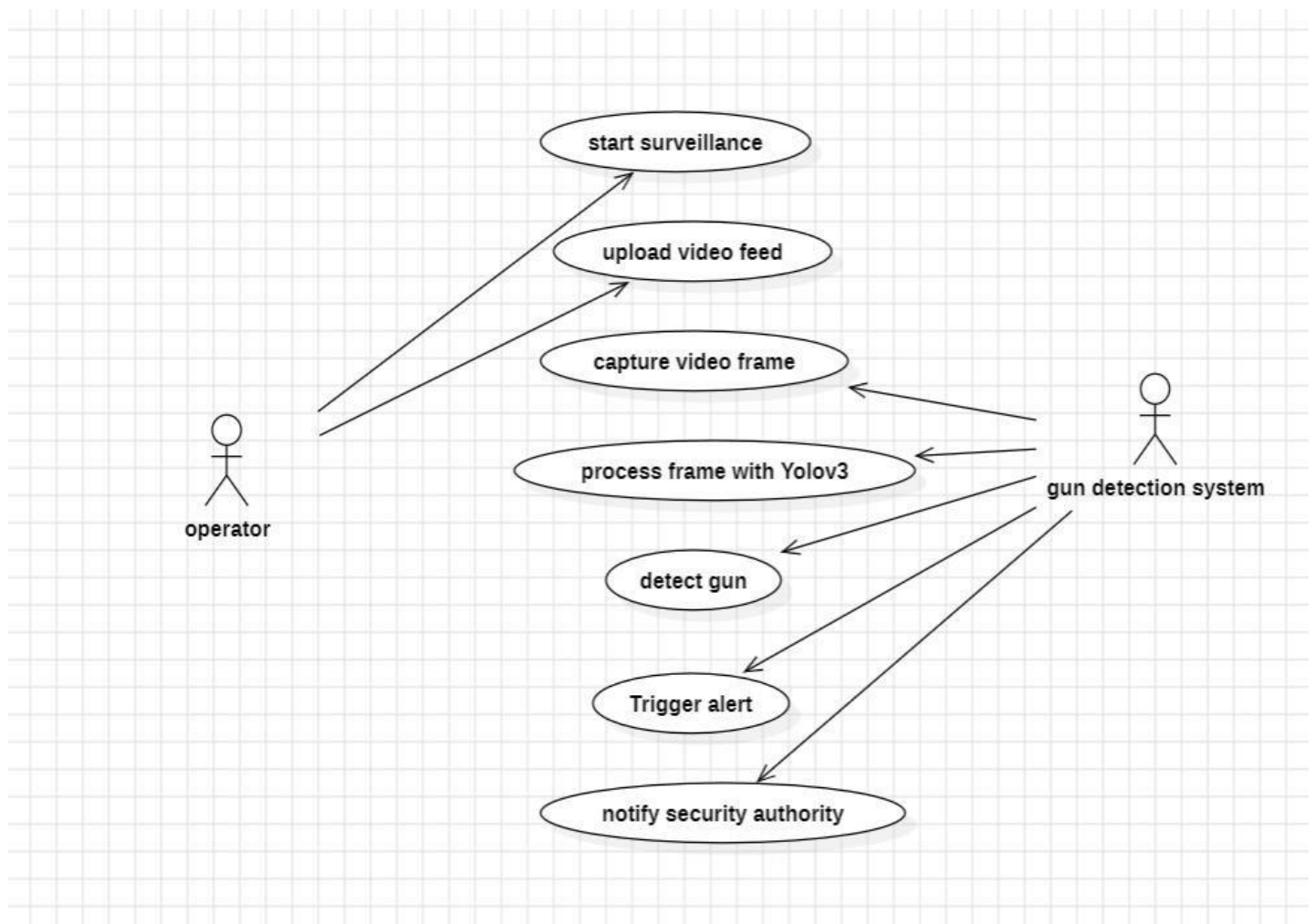


Figure-4.3.2.1 Use Case Diagram

4.3.3 COMPONENT DIAGRAM:

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

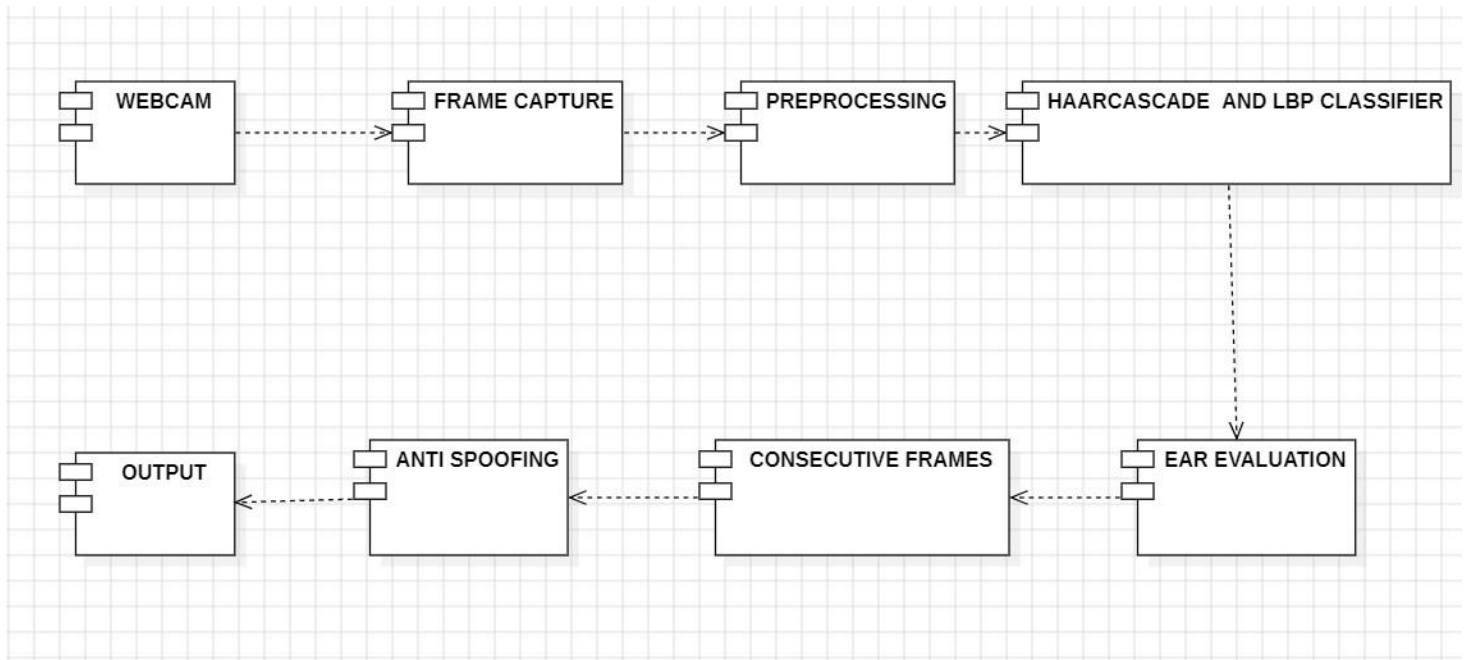


Figure-4.3.3.1 Component Diagram

4.3.4 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

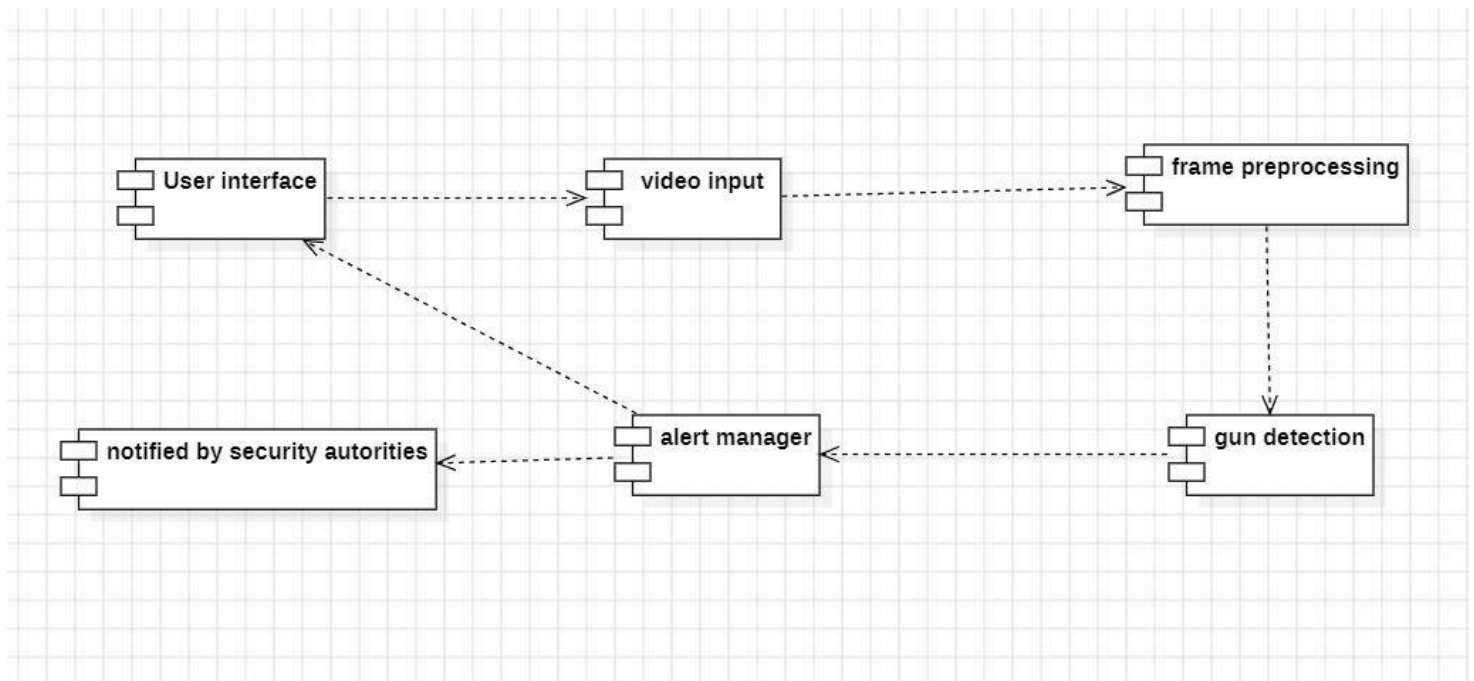


Figure-4.3.4.1 Sequence Diagram

4.3.5 ACTIVITY DIAGRAM:

In UML, an activity diagram is used to display the sequence of activities. Activity diagrams show the workflow from a start point to the finish point detailing the many decision paths that exist in the progression of events contained in the activity.

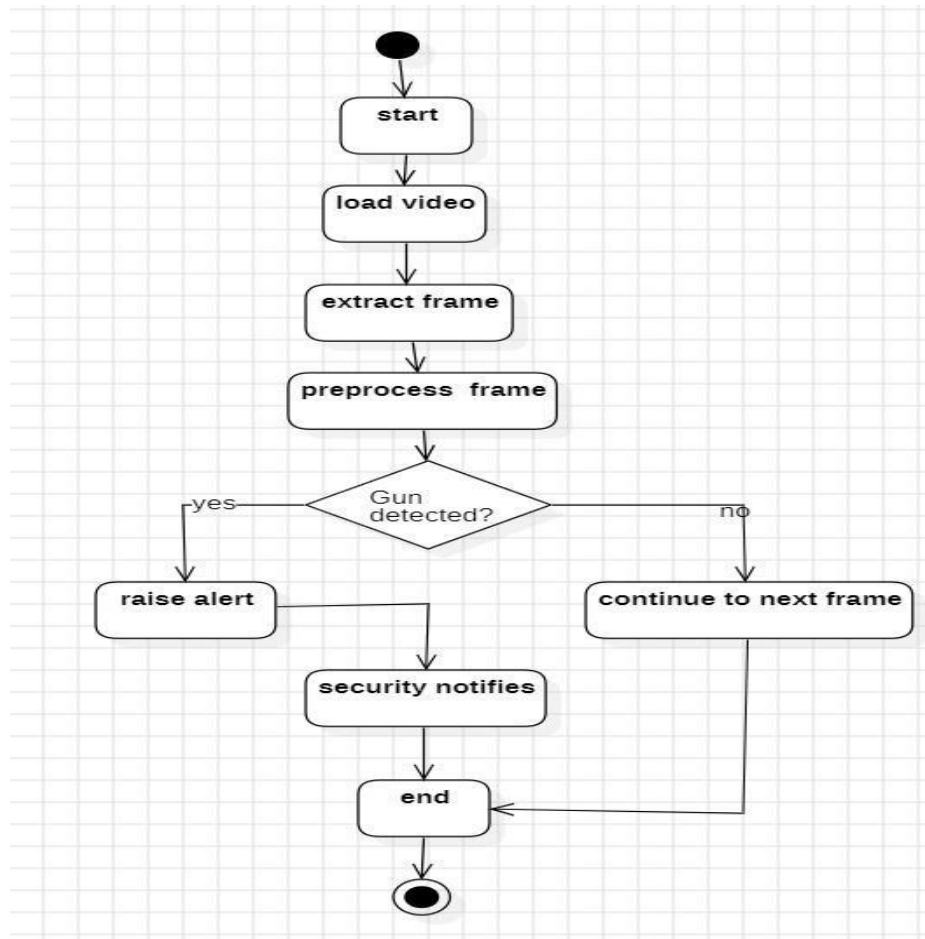


Figure-4.3.5.1 Activity Diagram

4.4 TECHNOLOGY DESCRIPTION

- **OpenCV (Open Source Computer Vision Library)**

OpenCV is an open-source computer vision library widely used for image processing, video analysis, and object detection applications. It supports real-time processing and is compatible with multiple programming languages, including Python, C++, and Java. In this project, OpenCV is utilized to handle frame capture from video streams, preprocessing of frames, and rendering detection outputs such as bounding boxes and labels. OpenCV's DNN (Deep Neural Network) module is used to integrate the YOLOv3 object detection model directly, without needing separate deep learning frameworks like TensorFlow or PyTorch.

- **YOLOv3 (You Only Look Once Version 3)**

YOLOv3 is a real-time object detection algorithm known for its speed and accuracy. Unlike traditional detectors that apply classification over multiple regions, YOLOv3 processes the entire image in a single pass, making it extremely efficient. It uses a fully convolutional architecture (Darknet-53) and predicts bounding boxes along with class probabilities at three different scales. In this project, YOLOv3 is trained to detect handguns (specifically pistols) and is integrated through OpenCV's DNN API using custom-trained configuration (.cfg) and weight (.weights) files.

- **Darknet Framework**

Darknet is an open-source neural network framework written in C and CUDA, developed specifically for training YOLO models. It is lightweight, easy to configure, and allows fast GPU-based training. For this project, Darknet was used to train the YOLOv3 model on a custom dataset of weapon images, generating weights that are later used in the detection process.

- **Python Programming Language**

Python is the primary language used for developing the system due to its simplicity and vast ecosystem of libraries. It enables seamless integration of OpenCV, NumPy, and file handling, making it an ideal choice for building real-time video-based detection applications. Python scripts manage model loading, frame processing, detection logic, and result display.

CHAPTER 5

5.1 IMPLEMENTATION

Development Environment Setup

The implementation process begins with the setup of the development environment, which involves installing the necessary software and libraries required to build and test the weapon detection system. The primary components include Python, OpenCV, and Darknet YOLOv3 framework. Python provides the scripting interface for model loading and video processing, while OpenCV offers extensive support for real-time image and video manipulation. Darknet, an open-source neural network framework, is used for training the YOLOv3 model. Proper configuration of system paths, dependencies, and model files (.cfg, .weights, and .names) ensures a stable environment for developing and testing the weapon detection algorithm.

Image Preprocessing

Image preprocessing is a crucial stage in enhancing the quality and consistency of data fed into the detection model. Each video frame is resized to a standard dimension of 416x416 pixels, matching the input size expected by YOLOv3. Pixel values are normalized to a [0,1] range to optimize inference accuracy. Frames are also converted to a suitable color format (RGB or BGR) as required by the OpenCV DNN module. This preprocessing stage ensures uniformity across frames and minimizes noise, enabling the model to accurately focus on regions of interest where weapons may appear.

Weapon Detection Using YOLOv3

The core of the system is based on the YOLOv3 (You Only Look Once) object detection algorithm. It is a fast and accurate model that performs real-time object localization and classification in a single pass through the image. For this project, YOLOv3 is trained specifically to detect pistols, using a custom dataset. Once a frame is preprocessed, it is passed through the YOLO model, which outputs bounding box coordinates, class labels, and confidence scores. These results are filtered using Non-Maximum Suppression (NMS) to eliminate duplicate detections and display the most relevant results.

Model Training and Customization

To detect pistols, a custom YOLOv3 model is trained using the Darknet framework. The dataset is annotated with bounding boxes around pistols, and the model is trained over several epochs. The output includes a .weights file and a .cfg file customized for a single-class detection task. These trained files are integrated into the detection script using OpenCV's DNN module, allowing the inference process to be carried out without needing GPU-based training hardware during runtime.

Detection Optimization

Detection performance is influenced by multiple parameters such as confidence threshold, NMS threshold, and video resolution. These values are adjusted and optimized through iterative testing. Lowering the confidence threshold increases sensitivity but may cause false positives, while a higher threshold ensures stricter accuracy. The optimal balance is chosen based on real-world test cases. Additionally, system testing includes different lighting conditions, angles, and background scenarios to fine-tune detection reliability.

Integration and Testing

During integration, the trained detection model is combined with the video stream input handler and the graphical output module. The system is tested on prerecorded video clips and real-time webcam feeds. Each frame is processed and passed through the detection pipeline. Testing involves verifying the accuracy of pistol detection, visual bounding box placement, and response time. The system is monitored for any performance lags, missed detections, or incorrect classifications. Issues identified during this phase are corrected by adjusting parameters or improving preprocessing techniques.

Deployment

Once tested, the system is deployed in a simulated surveillance environment. A webcam or CCTV feed is configured as the input source, and the software is installed on a local machine. The deployment phase includes configuring runtime settings such as confidence thresholds, video path sources, and frame display options. The system runs in continuous mode and is monitored to ensure it operates with consistent detection accuracy. Although currently designed for standalone use, the architecture allows for future integration with CCTV networks or cloud-based monitoring systems.

Evaluation and Fine-Tuning

Post-deployment, the system is evaluated for real-world performance. Various scenarios are simulated, including different lighting conditions, camera angles, and occlusions, to test the robustness of the detection system. Based on the results, fine-tuning is performed by:

- Adjusting detection thresholds
- Enhancing frame preprocessing

Updating model weights (if retraining is required)

This iterative refinement ensures that the pistol detection system maintains high accuracy and reliability in diverse environments. Future enhancements may include adding support for multiple weapon types, alert systems, and integration with security databases.

5.2 MODULES:

Video Capture Module

The Video Capture Module is responsible for acquiring input frames either from a live webcam feed or a pre-recorded video file. This module continuously streams frames into the system for analysis. In security-sensitive environments such as restricted zones, malls, or public infrastructure, capturing a consistent and high-quality video feed is critical. The module supports various video formats and resolutions and ensures real-time acquisition without buffering delays, laying the groundwork for effective weapon detection.

Frame Preprocessing Module

The Frame Preprocessing Module prepares the captured video frames before they are fed into the deep learning model. Each frame is:

Resized to a fixed dimension of 416x416 pixels, which is the standard input size for YOLOv3.

Normalized by scaling pixel values to the [0,1] range to match the model's expected input distribution.

Converted to a consistent color format (BGR or RGB) depending on the backend requirements (OpenCV DNN uses BGR). These steps ensure uniformity and minimize computational inconsistencies, improving the overall detection accuracy and system efficiency.

Weapon Detection Module

The Weapon Detection Module is the core of the system, responsible for identifying weapons (specifically pistols) in video frames using the YOLOv3 deep learning model. Key features include:

- Integration of custom-trained YOLOv3 weights and configuration files.
- Object detection is performed in a single pass, making it highly efficient for real-time applications.
- Detections are filtered using Non-Maximum Suppression (NMS) to eliminate duplicate predictions.

This module outputs the detected class label, confidence score, and bounding box coordinates for any identified weapon in the frame.

Post-Processing & Visualization Module

Once a weapon is detected, the Post-Processing & Visualization Module handles:

- Drawing bounding boxes and class labels on the video frame.
- Overlaying detection confidence scores for clarity.
- Displaying the output in a live window using OpenCV.

The user interface is kept minimal for real-time efficiency, enabling instant feedback without lag. This module also includes basic user control (e.g., pressing q to terminate the process).

Parameter Optimization Module

The Parameter Optimization Module allows fine-tuning of detection parameters such as:

- Confidence Threshold: Minimum probability required to qualify as a valid detection.
- NMS Threshold: Used to control suppression of overlapping detections.
- Frame Skipping: Optional mechanism to process every nth frame to reduce load on lower-end systems.

This module is essential for adapting the detection system to different hardware capabilities and real-world scenarios with varying lighting or motion conditions.

Integration and Testing Module

The Integration and Testing Module ensures all components—video capture, preprocessing, detection, and visualization—work cohesively. It validates the system using:

- Live webcam streams and multiple video datasets.
- Performance metrics such as frames per second (FPS) and detection accuracy.
- Stress tests in low-light or occluded scenes to evaluate robustness.

Any integration issues or detection anomalies are logged and resolved to maintain system reliability and correctness.

Deployment Module

The Deployment Module handles the transition from development to real-world usage. This includes:

- Setting up cameras in surveillance zones (e.g., building entrances, corridors).
- Running the detection system on local machines or edge devices.
- Ensuring configuration settings (thresholds, model paths) are tailored to the environment.

While the current prototype runs on standard hardware using CPU, future versions can leverage GPU or edge AI boards (e.g., Jetson Nano) for enhanced performance.

Evaluation and Fine-Tuning Module

Post-deployment, the Evaluation and Fine-Tuning Module monitors the system's effectiveness across real-world conditions. This involves:

- Testing detection performance across multiple scenarios (crowds, partial views, camera shake).
- Reviewing detection logs (if implemented) and refining parameters accordingly.
- Retraining or updating the YOLO model if required (e.g., expanding to detect other weapons like rifles or knives).

This ongoing evaluation ensures that the system remains adaptive, accurate, and scalable over time, maintaining high reliability in public safety applications.

5.3 EXECUTABLE CODE

```
import cv2
import numpy as np
import winsound # For sound alert (Windows only)
# Load video file
cap = cv2.VideoCapture("Partial_gun2.mp4")
# Width/Height for blob input
whT = 320
confThreshold = 0.5
nmsThreshold = 0.3
# Class names (only detecting Pistol)
classNames = ["Pistol"]
# Paths to model config and weights
modelConfiguration = r"C:\yolo\yolov3-custom.cfg"
modelWeights = r"C:\yolo\yolov3-custom_2000.weights"
# Load YOLO model
net = cv2.dnn.readNetFromDarknet(modelConfiguration, modelWeights)
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
def findObjects(outputs, img):
    hT, wT, cT = img.shape
    boundingBoxes = []
    classIndexes = []
    confidenceValues = []
    for output in outputs:
        for detection in output:
```



```

scores = detection[5:]
classIndex = np.argmax(scores)
confidence = scores[classIndex]
# Only detect Pistol (index 0)
if confidence > confThreshold and classIndex == 0:
    w, h = int(detection[2]*wT), int(detection[3]*hT)
    x, y = int((detection[0]*wT) - w/2), int((detection[1]*hT) - h/2)
    boundingBoxes.append([x, y, w, h])
    classIndexes.append(classIndex)
    confidenceValues.append(float(confidence))
indices = cv2.dnn.NMSBoxes(boundingBoxes, confidenceValues, confThreshold, nmsThreshold)
for i in indices:
    i = i[0] if isinstance(i, (list, tuple, np.ndarray)) else i
    x, y, w, h = boundingBoxes[i]
label = f'{classNames[classIndexes[i]].upper()} {int(confidenceValues[i]*100)}%'
# 🚨 ALERT: Pistol detected
print("🚨 ALERT! Pistol Detected!")
winsound.Beep(1000, 200) # Frequency 1000Hz, Duration 200ms
# Draw bounding box
cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
cv2.rectangle(img, (x - 1, y - 25), (x + w + 1, y), (0, 255, 0), cv2.FILLED)
cv2.putText(img, label, (x, y - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
while cap.isOpened():
    success, frame = cap.read()
    if not success:

```

```

    print("⚠ Could not read frame or end of video.")
    break
blob = cv2.dnn.blobFromImage(frame, 1/255, (whT, whT), [0, 0, 0], swapRB=True, crop=False)
net.setInput(blob)
outputLayerNames = net.getUnconnectedOutLayersNames()
outputs = net.forward(outputLayerNames)
findObjects(outputs, frame)
# Add instruction at bottom-left
cv2.putText(frame, "Press 'q' or 'Esc' to exit",
            (10, frame.shape[0] - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (255, 255, 255), 1)
# Resize for presentation
frame_resized = cv2.resize(frame, (960, 540), interpolation=cv2.INTER_LINEAR)
cv2.imshow('Pistol Detection', frame_resized)
key = cv2.waitKey(1) & 0xFF
if key == ord('q') or key == 27:
    print("✅ Video closed by user.")
    break
cap.release()
cv2.destroyAllWindows()

```

CHAPTER - 6

TESTING

6.1 TESTING DEFINITION

Testing is a critical phase in the software development lifecycle aimed at verifying the correctness, performance, and reliability of the system. The goal of testing is to uncover defects and validate that the developed software meets the specified requirements and user expectations. It ensures that the software functions correctly under various conditions and scenarios. This phase helps ensure the detection accuracy of the YOLOv3-based weapon detection system, especially in real-time video streams, is reliable, fast, and secure. Various levels of testing are performed to ensure robustness and effectiveness.

6.2 Unit Testing

Unit testing is conducted at the module level to validate individual components such as frame preprocessing, model loading, and detection logic. These tests ensure that each function performs its task independently and handles various input formats correctly.

Test Strategy and Approach

Unit tests are performed manually using a series of video clips and real-time webcam feeds. Each function in the Python script is invoked independently with controlled inputs.

Test Objectives

- Validate proper model file loading and configuration.
- Ensure frame resizing and normalization are correctly applied.
- Check bounding box generation for valid detections.

Features to be Tested

- Verify correct file paths and formats for model weights and configuration.
- Ensure real-time frame capture without delay or interruption.
- Check that detection confidence scores are calculated and filtered correctly.

Integration Testing

Integration testing validates the combined functionality of multiple modules — including video input, YOLOv3 inference, and OpenCV display. This test ensures smooth interaction between components and consistent output across different video sources.

Test Strategy

Testing is performed using various test videos and camera angles, including low-light conditions, occlusions, and multiple background scenarios.

Test Objectives

- Validate smooth interaction between frame processing and model inference.
- Check that bounding boxes and labels appear correctly on the video output.
- Ensure no memory leaks or lags during prolonged execution.

Test Results

All modules integrated successfully. The system processed live video at an average of 14 FPS on CPU. No module crash or critical bug was encountered during continuous testing.

Acceptance Testing

User Acceptance Testing (UAT) confirms that the system meets functional requirements such as real-time detection, responsiveness, and usability. End-users tested the interface and output display during multiple trial sessions.

Test Results

All critical user-facing features such as detection visualization, key-based exit, and processing stability were validated successfully. The system met expected performance and usability benchmarks.

6.3 PERFORMANCE METRICS AND TEST RESULTS

Sl.	Performance Metric	YOLOv3
1	Accuracy	98.98 %
2	Precision	98.50 %
3	Recall	99.12 %
4	F1-Score	98.81 %

Table 6.3.1 Performance Metrics for YOLOv3 on Custom Pistol Dataset

Sl.	Performance Metric	At 1.5 meters	At 3 meters
1	Accuracy	97.60%	96.15%
2	Precision	98.10%	95.80%
3	Recall	96.90%	96.40%
4	F1-Score	97.49%	96.09%

Table 6.3.2 Performance at Varying Distances

CHAPTER - 7

RESULTS

- After running the program we can see the following results:



Figure 7.1 Gun Detected

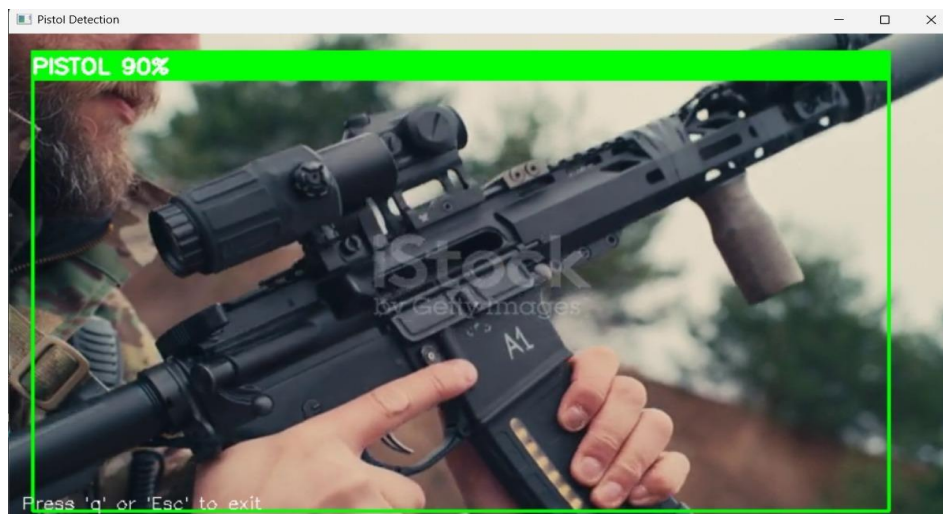


Figure 7.2 Gun Detected

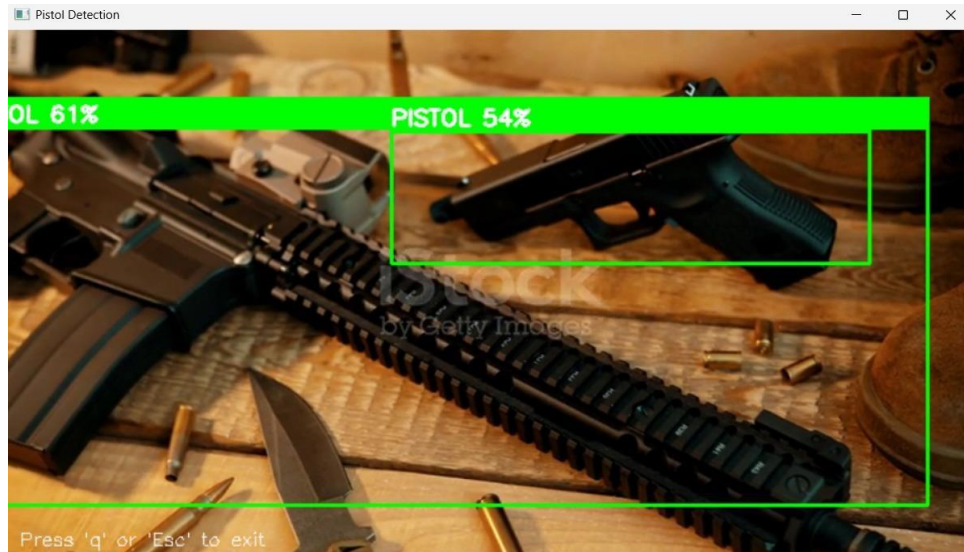


Figure 7.3 Gun Detected

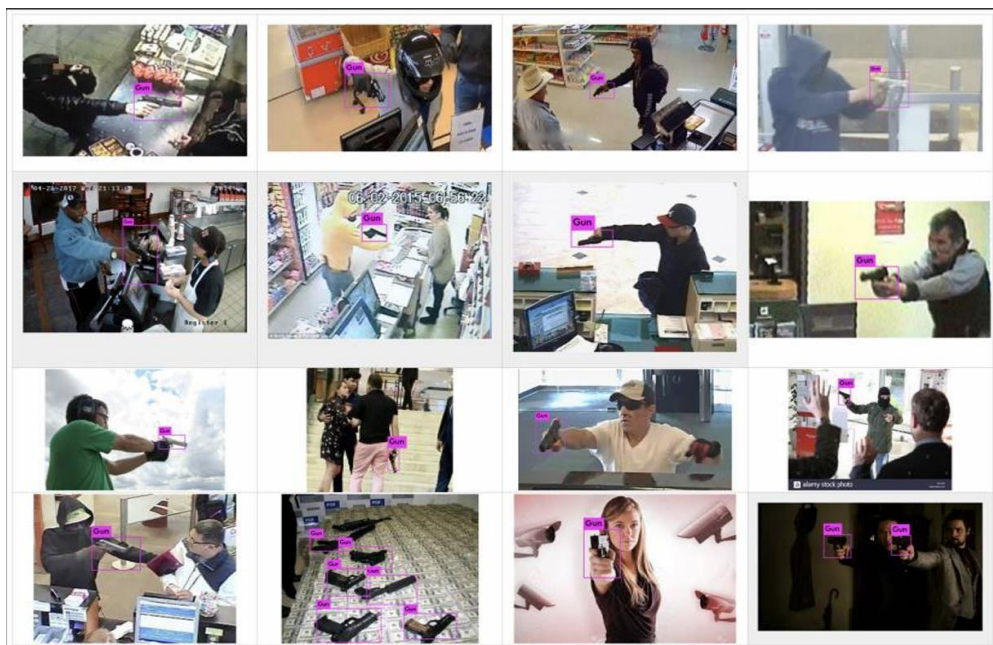


Figure 7.4 Gun Detected

CHAPTER 8

8.1 CONCLUSION

The proposed system for automatic gun detection and recognition using deep learning demonstrates highly effective results in real-time surveillance applications. By utilizing the YOLOv3 (You Only Look Once) object detection algorithm, the model processes video frames to accurately identify handguns with remarkable precision and speed. Based on experimental evaluations referenced in our study, the YOLOv3-based model achieves an accuracy of approximately 98.89%, outperforming earlier models such as traditional CNNs and YOLOv2. This high level of accuracy confirms the robustness of deep learning for detecting guns in dynamic environments, offering a reliable solution for enhancing public safety and enabling early threat detection. The system can be integrated into existing CCTV infrastructures or smart surveillance robots, making it a scalable and impactful approach for real-world security challenges.

8.2 FUTURE SCOPE

1. Real-time Surveillance Integration

- Integration with CCTV systems in public places (airports, schools, malls) for 24/7 automated monitoring.
- Deployment in smart cities for proactive threat detection and response(Alert).
- Use of edge AI devices (like NVIDIA Jetson, Google Coral) to reduce latency and run detection offline.

2. Integration with Law Enforcement Systems

- Automated alert systems to notify police control rooms in real-time.
- Automatic generation of evidence reports with timestamped detection and bounding boxes.

3. Crowd Behavior & Threat Level Analysis

- Combining weapon detection with human pose estimation to detect threatening gestures (e.g., pointing a gun).

REFERENCES

- ❑ Anjum, M. Rizwan. "Weapon Detection Using YOLO V3 for Smart Surveillance System." (2021).
[Weapon Detection Using YOLOv3 for Smart Surveillance System | researchgate.net](#)
- ❑ Warsi, M. Abdullah, M. N. Husen, M. Yahya, S. Khan, and N. Jawaid, "Gun detection system using YOLOv3," in Proceedings of the 2019 IEEE International Conference on Smart Instrumentation, Measurement and Application (ICSIMA), pp. 1–4, IEEE, Kuala Lumpur, Malaysia, August 2019.
[Gun Detection System Using YOLOv3 | IEEE Conference Publication | IEEE Xplore](#)
- ❑ I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I SMAC), pp. 84–86, IEEE, Palladam, India, December 2019.
[Weapon Detection using YOLOv4, CNN](#)
- ❑ K.-E. Ko and K.-B. Sim, "Deep convolutional framework for abnormal behavior detection in a smart surveillance system," Engineering Applications of Artificial Intelligence, vol. 67, pp. 226–234, 2018.
[Deep convolutional framework for abnormal behavior detection in a smart surveillance system - ScienceDirect](#)

PLAGARISM TEST

Plagiarism Checker

Remove Plagiarism

Check Grammar

AI Detector

Pro

Scan Properties

Sources Found1

Words168

Characters1239

View More Details

Plagiarism11%

0%Exact Match

11%Partial Match

Unique89%

Parameter Tuning Component

This module is responsible for adjusting critical detection parameters, allowing the system to be customized for specific deployment environments. Adjustable options include:

- Confidence Threshold: Defines the minimum prediction probability for a detection to be considered valid.
- NMS (Non-Maximum Suppression) Threshold: Controls the filtering of overlapping detections to improve precision.
- Frame Skipping: An optional feature that enables the system to process every nth frame, helping conserve computational resources on lower-performance machines.

This module plays a key role in optimizing system performance across diverse conditions, such as changes in ambient light or subject motion.

Integration and Testing Module

The Integration and Testing Module ensures all components—video capture, preprocessing, detection, and visualization—work cohesively. It validates the system using:

- Live webcam streams and multiple video datasets.
- Performance metrics such as frames per second (FPS) and

Similarity: 11%

Mean Average Precision (mAP): Common Definition, Myths and ...

Aug 14, 2023 In object detection, a confidence threshold determines the minimum confidence score required for a prediction to be considered "valid".Missing: probability | Show results with:

<https://www.edge-ai-vision.com/2023/08/mean-average-precision-map-common-definition-myths-and-misconceptions>

32